Практическая работа

Отчёт по практической работе № 5

Безопасная обработка пользовательского ввода

Дисциплина:	Основы разработки защищённого программного обеспечения и компьютерных сетей
Вариант:	19 — Система учёта транспортных средств организации
Исходная версия:	v0.4
Версия работы:	v0.5 — Secure Input Validation
Студент:	Мостовая Полина Андреевна
Группа:	ИУСбд-02-23

2026

Содержание

1 Тема и цель работы 2

2 Индивидуальный вариант и проект 2

3 Спецификация валидации 2

4 Схема обработки сообщения 2

5 Реализация 3

5.1 Модель и обработка сообщения 3

5.2 CLI 6

6 Таблица тестовых сценариев 6

6.1 Дополнительные проверки варианта 19 7

7 Фактические проверки 7

8 Обнаруженные и исправленные ошибки 9

9 Анализ Path Traversal и Command Injection 9

10 Соответствие требованиям 9

11 Ограничения прототипа 10

12 Ответы на контрольные вопросы 10

13 Вывод 11

14 Источники 11

1. Тема и цель работы

Цель работы — разработать и проверить централизованную безопасную проверку входных данных до передачи в бизнес-логику. Реализация продолжает проект v0.4 и сохраняет предметные валидаторы транспортных средств.

2. Индивидуальный вариант и проект

Вариант 19 требует нового логического типа вложения. Добавлен точный allowlist-ключ `vehicle_photo` для обозначения фотографии транспортного средства. Это идентификатор типа, а не путь и не имя файла. Загрузка и хранение изображений не входят в реализацию.

3. Спецификация валидации

Вход — UTF-8 JSON bytes размером не более 8192 байт. Обязательные поля: `title` (5–100 символов), `description` (20–2000), `category` (network/software/access/other), `priority` (строгое целое 1–5). Необязательны `reporter_ip` (IPv4/IPv6/null) и `attachment_key` (log/screenshot/report/vehicle_photo/null). Неизвестные и повторные ключи запрещены. Текст нормализуется NFC, внешние пробелы удаляются; CRLF описания нормализуется в LF. Управляющие символы запрещены, кроме перевода строки в описании.

HTML-подобный ввод сохраняется как текст. При выводе в HTML требуется отдельное контекстное экранирование. NFC объединяет канонически эквивалентное представление, но не подменяет символы по совместимости, как NFKC.

4. Схема обработки сообщения

Схема `docs/input_validation_flow.mmd` показывает получение bytes, лимит размера, UTF-8, JSON-разбор, структуру, нормализацию и allowlist. Отказы на каждом этапе безопасны. ТВ-01 — существующая граница между недоверенным вводом и приложением; проверенная модель передаётся дальше только после всех проверок.

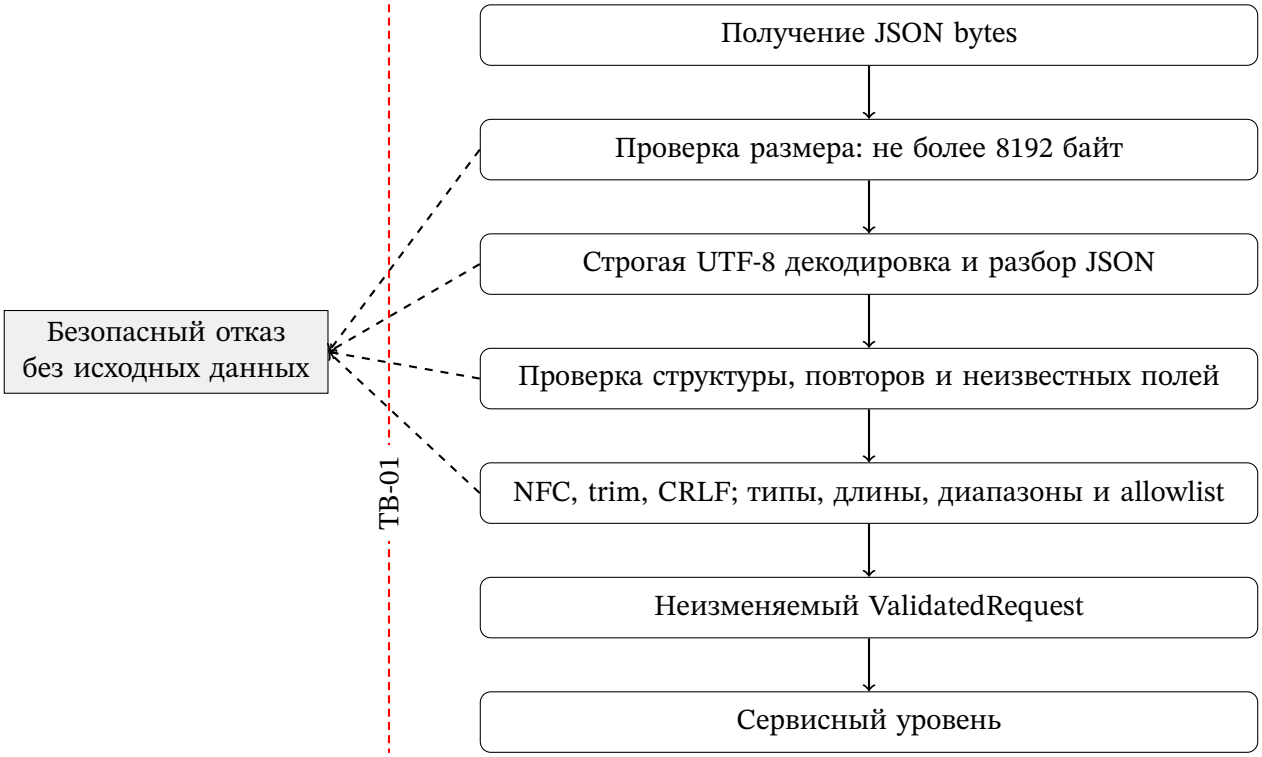


Рис. 1: Проверка на доверительной границе TB-01 до передачи модели дальше

5. Реализация

Центральный модуль возвращает неизменяемую типизированную модель. CLI ограничивает чтение до 8193 байт и отдаёт короткий JSON-ответ без исходного содержимого и traceback.

5.1. Модель и обработка сообщения

```
"""Безопасная проверка JSONобращений- до передачи в бизнеслогику-."""

from __future__ import annotations

import ipaddress
import json
import unicodedata
from dataclasses import dataclass

MAX_REQUEST_BYTES = 8192
ALLOWED_CATEGORIES = frozenset({"network", "software", "access", "other"})
ALLOWED_ATTACHMENTS = frozenset({"log", "screenshot", "report", "vehicle_photo"
    })
REQUIRED_FIELDS = frozenset({"title", "description", "category", "priority"})
OPTIONAL_FIELDS = frozenset({"reporter_ip", "attachment_key"})


class ValidationError(ValueError):
    """Контролируемая ошибка с безопасными полями для внешнего ответа."""

    def __init__(self, field: str, message: str) -> None:
        self.field = field
```

```
        self.message = message
        super().__init__(message)


@dataclass(frozen=True, slots=True)
class ValidatedRequest:
    """Неизменяемое обращение, прошедшее все проверки входного контракта."""

    title: str
    description: str
    category: str
    priority: int
    reporter_ip: str | None = None
    attachment_key: str | None = None


def _reject_constant(_: str) -> None:
    raise ValueError


def _unique_object(pairs: list[tuple[str, object]]) -> dict[str, object]:
    result: dict[str, object] = {}
    for key, value in pairs:
        if key in result:
            raise ValueError
        result[key] = value
    return result


def _text(value: object, field: str, minimum: int, maximum: int, *, multiline:
    bool) -> str:
    if type(value) is not str:
        raise ValidationError(field, "Ожидается строка")
    value = value.replace("\r\n", "\n").replace("\r", "\n") if multiline else
    value
    value = unicodedata.normalize("NFC", value).strip()
    for char in value:
        category = unicodedata.category(char)
        if category.startswith("C") and not (multiline and char == "\n"):
            raise ValidationError(field, "Недопустимый управляющий символ")
        if char == "\n" and not multiline:
            raise ValidationError(field, "Перевод строки не разрешён")
    if not minimum <= len(value) <= maximum:
        raise ValidationError(field, "Длина значения вне допустимого диапазона")
    return value


def _parse_json(raw: bytes) -> dict[str, object]:
    if type(raw) is not bytes:
        raise ValidationError("request", "Ожидаются байты UTF-8")
    if len(raw) > MAX_REQUEST_BYTES:
        raise ValidationError("request", "Превышен максимальный размер
    сообщения")
    try:
        decoded = raw.decode("utf-8", errors="strict")
    except UnicodeDecodeError:
        raise ValidationError("request", "Сообщение должно быть в UTF-8") from
    None
    try:
        parsed = json.loads(
```

```
            decoded,
            object_pairs_hook=_unique_object,
            parse_constant=_reject_constant,
        )
    except (json.JSONDecodeError, ValueError, RecursionError):
        raise ValidationError("request", "Некорректный JSON") from None
    if type(parsed) is not dict:
        raise ValidationError("request", "Ожидается JSONобъект-")
    return parsed


def validate_request(raw: bytes) -> ValidatedRequest:
    """Разобрать JSON bytes, проверить контракт и вернуть только проверенную
    модель."""
    data = _parse_json(raw)
    missing = REQUIRED_FIELDS - data.keys()
    if missing:
        field = sorted(missing)[0]
        raise ValidationError(field, "Отсутствует обязательное поле")
    unknown = data.keys() - REQUIRED_FIELDS - OPTIONAL_FIELDS
    if unknown:
        raise ValidationError("request", "Обнаружено неизвестное поле")

    title = _text(data["title"], "title", 5, 100, multiline=False)
    description = _text(data["description"], "description", 20, 2000, multiline=
    True)
    category = _text(data["category"], "category", 1, 32, multiline=False).lower
    ()
    if category not in ALLOWED_CATEGORIES:
        raise ValidationError("category", "Недопустимая категория")

    priority = data["priority"]
    if type(priority) is not int or not 1 <= priority <= 5:
        raise ValidationError("priority", "Ожидается целое число от 1 до 5")

    reporter_ip: str | None = None
    if "reporter_ip" in data and data["reporter_ip"] is not None:
        value = data["reporter_ip"]
        if type(value) is not str:
            raise ValidationError("reporter_ip", "Ожидается IPадрес- или null")
        value = value.strip()
        try:
            if "%" in value:
                raise ValueError
            reporter_ip = str(ipaddress.ip_address(value))
        except ValueError:
            raise ValidationError("reporter_ip", "Некорректный IPадрес-") from
    None

    attachment_key: str | None = None
    if "attachment_key" in data and data["attachment_key"] is not None:
        value = data["attachment_key"]
        if type(value) is not str or value not in ALLOWED_ATTACHMENTS:
            raise ValidationError("attachment_key", "Недопустимый логический
    ключ вложения")
        attachment_key = value

    return ValidatedRequest(title, description, category, priority, reporter_ip,
     attachment_key)
```

5.2. CLI

```
"""CLI для ограниченного приёма и проверки JSON через стандартный ввод."""

import json
import sys
from dataclasses import asdict

from app.validation import MAX_REQUEST_BYTES, ValidationError, validate_request


def main() -> int:
    raw = sys.stdin.buffer.read(MAX_REQUEST_BYTES + 1)
    try:
        request = validate_request(raw)
    except ValidationError as error:
        print(json.dumps({"status": "rejected", "field": error.field, "message":
    error.message}, ensure_ascii=False))
        return 2
    print(json.dumps({"status": "accepted", "request": asdict(request)},
   ensure_ascii=False))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

6. Таблица тестовых сценариев

Каждая строка соответствует сценариям раздела 10.1 задания. Фактический результат записан по запуску pytest после реализации; таблица будет дополнена реальными скриншотами на следующем этапе.

Таблица 1: Обязательные сценарии и фактические результаты

№	Проверка	Ожидание	Фактический результат	Статус
1	Корректное обращение	Принято	Тест пройден	Пройдено
2	Отсутствует обязательное поле	Отказ	Тест пройден	Пройдено
3	Неизвестное поле	Отказ	Тест пройден	Пройдено
4	Короткий заголовок	Отказ	Тест пройден	Пройдено
5	Слишком длинное описание	Отказ	Тест пройден	Пройдено
6	Неизвестная категория	Отказ	Тест пройден	Пройдено
7	Приоритет меньше 1	Отказ	Тест пройден	Пройдено
8	Приоритет больше 5	Отказ	Тест пройден	Пройдено
9	Приоритет-строка	Отказ	Тест пройден	Пройдено
10	Приоритет True	Отказ	Тест пройден	Пройдено
11	Некорректный IP	Отказ	Тест пройден	Пройдено
12	Неизвестное вложение	Отказ	Тест пройден	Пройдено
13	Путь вместо ключа	Отказ	Тест пройден	Пройдено
14	Повреждённый JSON	Отказ	Тест пройден	Пройдено

№	Проверка	Ожидание	Фактический результат	Статус
15	Размер более 8192 байт	Отказ	Тест пройден	Пройдено
16	Повторяющиеся ключи	Отказ	Тест пройден	Пройдено
17	Управляющий символ	Отказ	Тест пройден	Пройдено
18	Корректный IPv6	Принято	Тест пройден	Пройдено
19	Канонически эквивалентный Unicode	NFC	Тест пройден	Пройдено
20	HTML-подобный текст	Сохранён как текст	Тест пройден	Пройдено

6.1. Дополнительные проверки варианта 19

Сценарий	Ожидание	Фактический результат	Статус
vehicle_photo	Принято	Тест пройден	Пройдено
Неизвестный тип	Отказ	Тест пройден	Пройдено
Имя файла vehicle_photo.jpg	Отказ	Тест пройден	Пройдено

7. Фактические проверки

Первичный системный `python3` оказался устаревшим и не поддерживает синтаксис проекта. Проверки выполнены штатным окружением проекта `./.venv/bin/python -m pytest -q`: получено 92 passed; ошибок и пропусков нет. Реальный CLI принял два корректных файла с кодом 0 и отклонил два некорректных с кодом 2. В коде `app/` нет вызовов системных команд. В тестах `subprocess.run` запускает CLI с постоянными аргументами, без shell, а пользовательские данные передаются через stdin. Результаты команд показаны на рисунках 2–6.

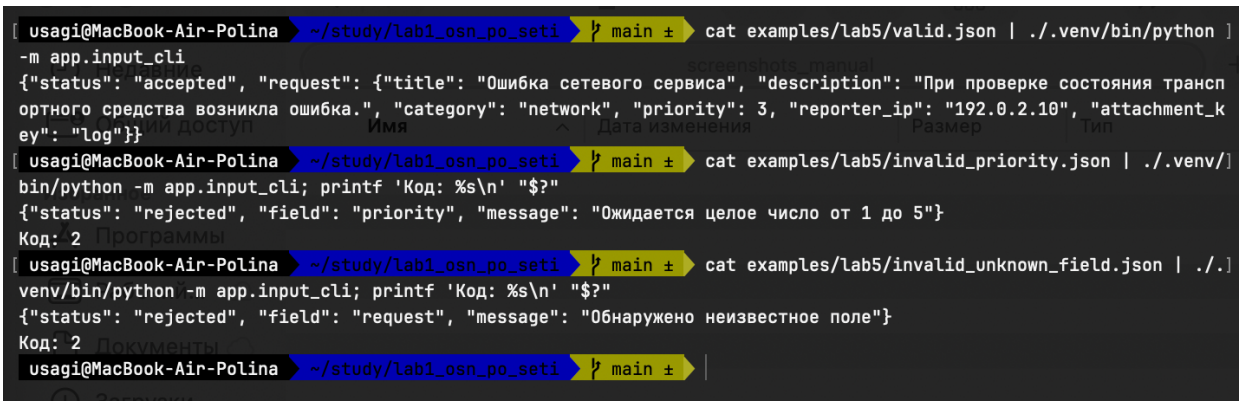


Рис. 2: Принятие корректного обращения и контролируемый отказ по приоритету и неизвестному полю.

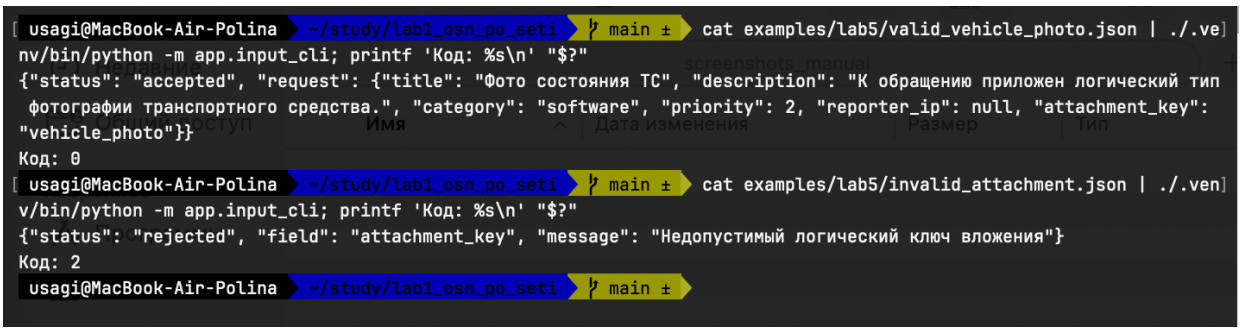


Рис. 3: Проверка логического ключа `vehicle_photo` и отклонение пути вместо ключа.



Рис. 4: Негативные проверки JSON, размера сообщения, дублирования ключей и глубокой вложенности.

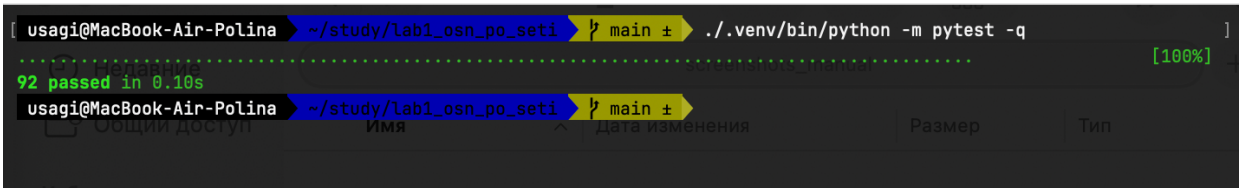


Рис. 5: Полный запуск автоматизированного набора: 92 теста пройдены.

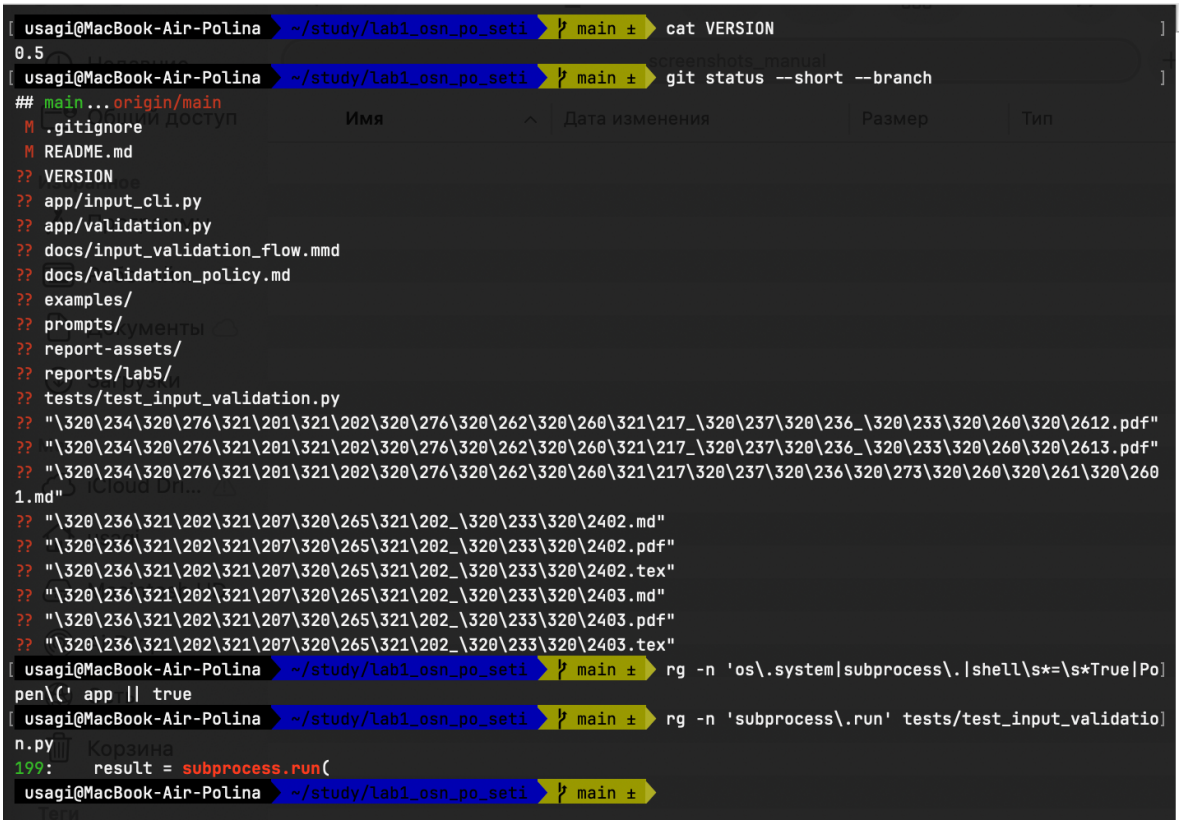


Рис. 6: Версия проекта, состояние рабочей копии и поиск вызовов команд ОС.

8. Обнаруженные и исправленные ошибки

Первый сбор тестов выявил два несвязанных вопроса: системный python3 не соответствовал версии Python проекта, а вспомогательная проверка отсутствия утечки некорректно сравнивала пустую строку. Использован проектный virtualenv и исправлено тестовое утверждение. После этого добавлены проверки граничных значений, глубокой вложенности и фактической CLI-точки. Ошибок реализации после повторной проверки не осталось; причины уязвимого поведения в исходном проекте этим тестированием не устанавливались.

9. Анализ Path Traversal и Command Injection

Пользовательское значение вложения проверяется сравнением с фиксированным allowlist и не используется в файловых операциях. В приложении не найдено `os.system`, `subprocess`, `Popen` или `shell=True`. В тесте интеграции безопасный `subprocess.run` запускает CLI с постоянными аргументами и без оболочки. Модуль не запускает системные команды.

10. Соответствие требованиям

Требование	Реализация	Проверка

Лимит и UTF-8	До JSON-разбора	Негативные тесты
JSON-структура и дубликаты	Строгий parser hook	Негативные тесты
Типы и диапазоны	`ValidatedRequest`	20 обязательных сценариев
Вариант 19	`vehicle_photo`	Три отдельных теста
Защита от path traversal	Ключ не является путём	Отказ пути/имени файла
Безопасная диагностика	Только имя поля и общий текст	Проверка отсутствия эха

11. Ограничения прототипа

CLI локальный и не является сетевым API; аутентификация, авторизация для заявок, база данных, файловое хранилище и загрузка изображения не реализованы. Лимит размера не ограничивает частоту запросов. Валидация не заменяет SQL-параметризацию или контекстное экранирование вывода.

12. Ответы на контрольные вопросы

1. Проверка устанавливает, соответствует ли вход контракту до его использования.
2. Проверяет доверенная сторона, потому что клиентские проверки можно обойти или изменить.
3. Синтаксическая проверка проверяет форму; семантическая — допустимый смысл и диапазон.
4. NFC помогает привести канонически одинаковые Unicode-строки к одному представлению.
5. Каноническое представление — согласованная форма значения для сравнения и обработки.
6. Строки являются данными; допустимость определяется контекстом и контрактом, а не поиском подозрительных подстрок.
7. Allowlist перечисляет допустимое; denylist перечисляет известное недопустимое и может пропускать новые варианты.
8. Ограничение поля не учитывает общий размер JSON, структуру и служебные накладные расходы.
9. В Python `bool` — подкласс `int`, поэтому используется проверка точного типа.
10. `ipaddress.ip_address()` проверяет IPv4/IPv6 и строит объект адреса.
11. Неизвестное поле может незаметно менять смысл операции или обходить предположения слоя приложения.
12. Разные JSON-парсеры могут выбрать разные значения дублирующегося ключа.
13. Путь из пользовательских данных может выйти за пределы разрешённого каталога.
14. Path Traversal — использование сегментов пути для обращения к файлу вне разрешённой области.
15. `shell=True` интерпретирует строку оболочкой и может превратить данные в команды.
16. Проверка входа не гарантирует безопасную сборку SQL; запросы следует параметризовать.
17. Валидация решает, допустимо ли значение; HTML-экранирование безопасно представляет его в конкретном контексте.

18. HTML-подобная строка может быть обычным текстом; удаление меняет данные и не заменяет безопасный вывод.
19. Негативное тестирование проверяет ожидаемый отказ на ошибочных и намеренно некорректных входах.
20. Fail Secure — при сбое отказать безопасно, не продолжая частично операцию.
21. Собственный тип ошибки позволяет отличать отказ контракта и безопасно формировать ответ.
22. Не раскрывают исходный запрос, секреты, внутренние пути и трассировку исключения.
23. Клиентскую проверку можно обойти; доверенный сервер проверяет заново.
24. Разделение данных и команд обеспечивают параметризация запросов, фиксированные аргументы и отказ от оболочки.
25. Соответствие показывают трассируемая политика, тесты ожидаемого результата и проверяемые фактические выводы.

13. Вывод

Создан отдельный валидатор JSON и CLI, сохраняющие прежнюю предметную валидацию автопарка. Контракт формирует неизменяемую модель, а вариант 19 расширяет список логических типов вложений ключом `vehicle_photo`. Тесты подтверждают только проверенные свойства прототипа; они не доказывают безопасность будущего веб-интерфейса или файлового хранилища.

14. Источники

1. Практическая работа №5 «Безопасная обработка пользовательского ввода», предоставленная методичка, 2026.
2. Документы проекта v0.2–v0.4: требования, модель угроз, реестр рисков и защищённая архитектура.
3. Python 3: стандартные модули `json`, `unicodedata`, `ipaddress`.